

Demystifying Random Number in Ethereum Smart Contract: Taxonomy, Vulnerability Identification, and Attack Detection

Peng Qian , Jianting He , Lingling Lu , Siwei Wu , Zhipeng Lu , Lei Wu , Yajin Zhou , and Qinming He 

Abstract—Recent years have witnessed explosive growth in blockchain smart contract applications. As smart contracts become increasingly popular and carry trillion dollars worth of digital assets, they become more of an appealing target for attackers, who have exploited vulnerabilities in smart contracts to cause catastrophic economic losses. Notwithstanding a proliferation of work that has been developed to detect an impressive list of vulnerabilities, the bad randomness vulnerability is overlooked by many existing tools. In this article, we make the first attempt to provide a systematic analysis of random numbers in Ethereum smart contracts, by investigating the principles behind pseudo-random number generation and organizing them into a taxonomy. We also lucubrate various attacks against bad random numbers and group them into four categories. Furthermore, we present *RNVulDet* – a tool that incorporates taint analysis techniques to automatically identify bad randomness vulnerabilities and detect corresponding attack transactions. To extensively verify the effectiveness of *RNVulDet*, we construct three new datasets: i) 34 well-known contracts that are reported to possess bad randomness vulnerabilities, ii) 214 popular contracts that have been rigorously audited before launch and are regarded as free of bad randomness vulnerabilities, and iii) a dataset consisting of 47,668 smart contracts and 49,951 suspicious transactions. We compare *RNVulDet* with three state-of-the-art smart contract vulnerability detectors, and our tool significantly outperforms them. Meanwhile, *RNVulDet* spends 2.98 s per contract on average, in most cases orders-of-magnitude faster than other tools. *RNVulDet* successfully reveals 44,264 attack transactions. Our implementation and datasets are released, hoping to inspire others.

Index Terms—Ethereum, smart contract, random number, vulnerability identification, attack detection, taint analysis.

Manuscript received 25 July 2022; revised 21 April 2023; accepted 22 April 2023. Date of publication 28 April 2023; date of current version 18 July 2023. This work was supported in part by the Key R&D Program of Zhejiang Province under Grant 2022C01086, and in part by the National Natural Science Foundation of China under Grants 62172360 and U21A20467. Recommended for acceptance by J. Sun. (Peng Qian and Jianting He are co-first authors.) (Corresponding author: Yajin Zhou.)

Peng Qian is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310018, China, and also with the Collaborative Innovation Center of Artificial Intelligence by MOE and Zhejiang Provincial Government, Hangzhou 310018, China (e-mail: messi.qp711@gmail.com).

Jianting He, Lingling Lu, Siwei Wu, Zhipeng Lu, Yajin Zhou, and Qinming He are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310018, China (e-mail: jiantinghe@zju.edu.cn; lulingling@email.cufe.edu.cn; wusw1020@zju.edu.cn; alexious@zju.edu.cn; yajin_zhou@zju.edu.cn; hqm@zju.edu.cn).

Lei Wu is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310018, China, and also with the Key Laboratory of Blockchain and Cyberspace Governance of Zhejiang Province, Hangzhou 310018, China (e-mail: lei_wu@zju.edu.cn).

Digital Object Identifier 10.1109/TSE.2023.3271417

I. INTRODUCTION

ETHEREUM has become one of the most prevalent blockchain platforms in recent years [1], [2], which is the first to enable the functionality of smart contracts. Ethereum smart contracts are runnable code encoding predefined rules and executing on the blockchain via the Ethereum Virtual Machine (EVM). Smart contracts make the automatic execution of contract terms possible, giving rise to a plethora of interesting decentralized applications (DApps), such as gambling game [3], lottery [4], decentralized finance [5], and distributed exchange [6].

Nowadays, smart contracts carry trillion dollars worth of digital assets, attracting numerous attacks [7], [8], [9]. Indeed, Ethereum has already encountered several devastating attacks on vulnerable smart contracts [10]. The most prominent ones were the DAO attack in 2016 [11] and the Parity Multisig Wallet attack in 2017 [12], together causing a loss of over \$400 million at that time. In 2018, the well-known game contract *Fomo3D* was compromised by hackers, who exploited the bad randomness vulnerability in the contract, leading to a loss of \$2 million [13]. Recently, cybercriminals hacked into the *Cream.Finance* contract, exploiting the reentrancy vulnerability to steal more than \$130 million worth of digital assets [14].

In response to these attacks, there have been surging research interests in detecting smart contract vulnerabilities [15], [16], [17], [18], [19], [20]. Existing methods can be roughly funneled into two categories. One line of work leverages static program analysis techniques [21], [22], [23], [24] (e.g., symbolic execution and abstract interpretation) to identify smart contract vulnerabilities. Another line of work [25], [26], [27] executes program traces dynamically and exposes potential vulnerabilities by monitoring runtime behaviors during execution. While existing efforts have identified a common set of vulnerabilities, one important category of vulnerabilities, i.e., *bad randomness vulnerability* [28], has been largely overlooked so far. A bad randomness vulnerability is an exploitable bug that attackers can manipulate or predict the generated pseudo-random number to gain benefits illegitimately. For instance, when a smart contract uses public on-chain information (e.g., block timestamp) as the random seed to trigger critical operations (e.g., asset transfer), malicious miners may manipulate the on-chain information to generate a pseudo-random number that is beneficial to them.

Why Focus on Bad Randomness Vulnerability. (1) A number of popular decentralized applications (such as *Universe-Galaxy* [29], *Fomo3D* [30], *CryptoKitties* [31], and *Dice2-Win* [32]) utilizes pseudo-random numbers to implement specific functions. Therefore, it is extremely important for such decentralized applications to ensure that the generated pseudo-random numbers are free of bugs. (2) Bad randomness vulnerability in Ethereum smart contracts has caused enormous financial losses (over \$30 million) [13], [33]. (3) Upon scrutinizing current vulnerability detection approaches for smart contracts, we empirically found that most existing tools solely support detecting a subfield of the bad randomness vulnerability such as block dependency [7], [17], [21], [26], while lacking an in-depth and comprehensive study on the perspective of bad randomness vulnerabilities. These facts motivate us to demystify random numbers in Ethereum smart contracts thoroughly as well as its vulnerability identification and attack detection.

In this article, we investigate the prevalence of random numbers in Ethereum smart contracts. To the best of our knowledge, this is the first work to provide a systematic analysis on the inner workings of various pseudo-random number generation strategies. In particular, we summarize four categories of attacks against weak random numbers. We also present concrete examples correspondingly to illustrate the attack procedure of each kind of attack. More importantly, we propose *RNVulDet* – a tool that adopts taint analysis techniques to automatically identify bad randomness vulnerabilities and detect corresponding attack transactions. *RNVulDet* simulates the runtime environment of the EVM and executes a smart contract through bytecode. During the simulated execution, *RNVulDet* recovers the control flow execution paths from the contract bytecode, followed by the taint analysis. Once the execution reaches the taint sink check, *RNVulDet* combines the predefined vulnerability patterns to determine whether the contract has a bad randomness vulnerability.

Specifically, we implement *RNVulDet* by advocating four key designs: (1) *Stack State Examination*. We design a stack state examination mechanism to reduce the execution of potential repeated program paths, alleviating the path explosion problem during the simulated execution. (2) *Memory Segmentation*. We introduce a memory segmentation-based alias analysis paradigm to deal with read and write dependencies in Memory. (3) *Storage Key-Value Pair Comparison*. We develop a key-value pair comparison technique to handle the data dependencies in Storage. (4) *Transaction Replay*. Finally, we engage in a transaction replay engine that can monitor the runtime states of a smart contract and hook into its execution to reveal random number attack instances.

To validate the usefulness of *RNVulDet*, we conduct extensive experiments, comparing with two static analyzers, i.e., *Slither* [16] and *Mythril* [34], and one dynamic tool, i.e., *ConFuziuss* [27], on three datasets. Experimental results confirm the effectiveness of *RNVulDet* in detecting bad randomness vulnerabilities and capturing corresponding attack transactions, i.e., 181 bad randomness vulnerabilities and 44,264 attack transactions have been identified, outperforming existing tools by a large margin. Moreover, *RNVulDet* achieves high efficiency in analyzing smart contracts, i.e., spending 2.98 s per contract on

average, in most cases orders-of-magnitude faster than other tools.

Our key contributions can be summarized as follows:

- To our knowledge, this is the first work to present a systematic analysis of random numbers in Ethereum smart contracts. Particularly, we identify common techniques used by pseudo-random number generation and organize them into a taxonomy. We also sum up four categories of attacks against bad random numbers.
- We present *RNVulDet*, a tool that automatically identifies bad randomness vulnerabilities in smart contracts and detects random number attack transactions. *RNVulDet* consists of four key components, i.e., stack state examination, memory segmentation, storage key-value pair comparison, and transaction replay.
- Extensive experiments on three datasets demonstrate that our proposed method is indeed useful in detecting bad randomness vulnerabilities and capturing random number attacks. *RNVulDet* surpasses state-of-the-art vulnerability detectors and overall provides interesting insights. To embrace the community, we have released the implementation of *RNVulDet* and datasets at <https://github.com/Messi-Q/RNVulDet>.

Paper Organizations: The rest of the article is organized as follows. In Section II, we briefly introduce the background of Ethereum, smart contract, Ethereum Virtual Machine, and taint analysis technique. Then, we present a fine-grained classification of pseudo-random number generation methods in Ethereum smart contracts and provide the taxonomy of random number attacks in Section III. In Section IV, we introduce the overall architecture and workflow of *RNVulDet*. Section V describes the dataset construction, and presents the evaluation results. In Sections VI and VII, we discuss possible future work and introduce related work. Finally, we conclude the study in Section VIII.

II. BACKGROUND

In this section, we outline the required background for understanding the settings of our work, including the descriptions of Ethereum, smart contract, Ethereum virtual machine, and taint analysis technique.

A. Ethereum and Smart Contract

Ethereum is one of the representative blockchain platforms, renowned for its introduction of smart contract functionality [35]. On the platform, there exist two types of accounts: *user* and *contract*, both of which store an Ether (i.e., the cryptocurrency of Ethereum) balance and publicly reside on the blockchain. User accounts, also known as externally owned accounts (EOAs), are managed by private keys, the 64-character hexadecimal string.

Contract accounts instead are controlled by code, which are known as smart contracts. A smart contract can implement arbitrary rules for manipulating digital assets, such as sending or receiving Ether. The defined rules in a smart contract will be strictly and automatically followed during execution,

effectuating the ‘code is law’ logic [19]. Explicitly, the code describes an agreement between the contract account and any users that interact with it. For example, such a contract program could encode the rules of a gambling game. Since the smart contract is stored on the blockchain, it becomes immutable and its execution is guaranteed by the blockchain. Ethereum smart contracts are usually developed using a high-level programming language, typically *Solidity*. Once deployed on the Ethereum, smart contracts will be broadcast to all bookkeeping nodes for backups.

B. Ethereum Virtual Machine

Ethereum defines a stack-based virtual machine dubbed Ethereum Virtual Machine (EVM) to determine the outcome of the execution of a smart contract. EVM is the runtime environment for the transaction execution in Ethereum. EVM embraces a set of exclusive instructions, including commonly used arithmetic operations and comparison operations. It also supports some blockchain-specific instructions like querying block numbers. The machine translates the contract bytecode into stack operations, where each operand either pops or pushes values to the stack, and each value has a 256-bit word size. In particular, EVM engages in three different structures to store data:

- **Stack.** EVM is a stack-based machine, and thus performs all computations in a data space termed Stack. All in-memory values are also stored in the stack. It has a maximum depth of 1,024 elements and supports a word size of 256 bits.
- **Memory.** Memory is a word-addressed byte array. Whenever the code execution of a contract is finished, the memory is cleared out. Write operations to the memory can be of 8 or 256 bits, whereas read operations are limited to 256-bit words.
- **Storage.** Storage is organized as a key-value store, which is the only way for smart contracts to retain state across executions. Data in Storage will be permanently persisted on the Ethereum world state¹. For instance, a gaming smart contract could leverage Storage to maintain the balance of each player.

To prevent the contract code to be stuck in endless loops of execution, the concept of *gas* has been introduced. It associates costs with the execution of every instruction. When issuing a transaction,² the sender has to specify the gas fee that he/she is willing to pay to the miner³ for the execution of the contract. The gas fee is computed by $gas_cost \times gas_price$. Gas cost depends on the computational resource taken by the execution, while the gas price is offered by the transaction creators [7]. Gas price denotes the number of Wei (1 Wei = 10e-18 Ether) to be paid per unit of gas for all computation costs incurred as

¹Ethereum world state refers to a data structure stored on the blockchain mapping addresses to account states.

²A transaction is a type of message call that serves three purposes, including transferring Ether, deploying a smart contract, and invoking functions of a smart contract [36].

³Smart contract transactions are executed by miners, which are a network of mutually distrusting nodes.

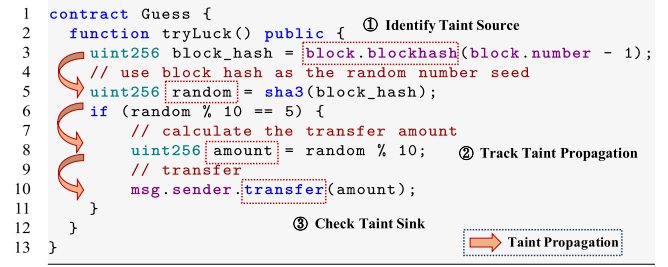


Fig. 1. An example of taint analysis in smart contracts.

a result of the transaction execution. Generally speaking, the higher the gas price set by the transaction creators, the sooner the miners will pack the transaction into the block. To limit gas cost, the sender who initiates a transaction will set the gas limit, which determines the maximum gas cost. If the gas cost of the transaction exceeds its gas limit, the execution will fail and throw an out-of-gas error [37]. In the meantime, EVM will terminate the execution and roll back to the previous state of the transaction. For a more thorough background of EVM, we refer the readers to the Ethereum yellow paper [36], where the formal specification and detailed inner workings of EVM can be found.

C. Taint Analysis for Vulnerability Detection

Taint analysis is one of the most classical program security checking techniques [38], [39]. During program execution, the taint analyzer reveals potential vulnerabilities in the code by tracking and checking the propagation trace of taint sources. Existing taint analysis techniques for program bug detection can be summarized in the following three key steps.

- **Identify Taint Source.** Taint sources are locations in the program where data is being read from a potentially risky source, like environment variables, command-line arguments, and file metadata. Therefore, the first key step for taint analysis is to identify such taint sources and mark them correspondingly.
- **Track Taint Propagation.** Intuitively, the marked taint sources may be assigned to other elements through data dependencies in the contract code during execution. To address this issue, the taint analyzer tracks the propagation flow of the taint sources and flags the elements affected by the taint sources accordingly.
- **Check Taint Sink.** From the perspective of taint analysis, the program is vulnerable if tainted elements endanger key locations, which are known as taint sinks. Every potential vulnerability has its own sinks. As such, the taint analyzer checks whether there is a trace that tainted elements can follow from taint sources to taint sinks, thereby exposing potential vulnerabilities.

Fig. 1 presents a simplified example to illustrate the taint analysis in smart contracts. First, taint analyzer $\mathbb{A}$ identifies the taint source, i.e., `block.blockhash` (line 3). Then, $\mathbb{A}$ tracks the path that the taint source travels, and marks tainted elements accordingly, i.e., `random` (line 5) and `amount` (line 8). Finally, $\mathbb{A}$ checks whether the `transfer` operation (line 10) is the taint

sink and exposes the bad randomness vulnerability. Note that we integrate three granularities of taint propagation in the bad randomness vulnerability analysis, *namely* the element-level in Stack, the memory segment-level in Memory, and the storage slot-level in Storage.

III. ETHEREUM RANDOM NUMBER

In this section, we first investigate the principles behind different pseudo-random number generation strategies in Ethereum smart contracts and organize them into a taxonomy accordingly. Meanwhile, we introduce several measure indicators to evaluate each pseudo-random number generation method. Finally, we present the general process of the random number attack. We also group the existing random number attack methods into four categories and deliver concrete examples to clarify each type of attack correspondingly.

A. Taxonomy of Random Number Generation

Definition 1 (Pseudo-Random Number). A pseudo-random number is a number generated using a set of random seeds and a mathematical algorithm that gives equal probability to all numbers occurring in the specified distribution.

After investigating existing pseudo-random number generation methods for smart contracts, we empirically found that current approaches can be roughly cast into two categories, namely using either on-chain information or off-chain resources to produce pseudo-random numbers⁴.

1) *Randomness Generated With On-Chain Information:* Our first insight is to use on-chain information to generate pseudo-random numbers for smart contracts. Since smart contracts are programs built on the blockchain, they embrace some dedicated built-in functions to gain access to on-chain information. As the on-chain information possesses certain randomness, current contract developers often treat them as random seeds. For example, `block.timestamp` which can obtain the block timestamp is usually treated as a cardinal seed to produce pseudo-random numbers. Typical on-chain information includes block information, environmental information, member variables of contracts, etc.

Block Information. Block information refers to the properties of blocks in the blockchain, such as block hash, block timestamp, block number, and block gas limit. Fig. 2 shows the relevant block information in Ethereum. Due to the randomness of block information, they are usually preferred as random seeds. A smart contract is able to gain such information of the current block where the transaction is taking place through designated instructions. However, malicious attackers can also access the block information with these instructions in an attempt to manipulate or predict the generated pseudo-random numbers. Another line of schemes attempts to use the block information of a future block as a basic seed, thus preventing attackers from predicting pseudo-random numbers. Unfortunately, miners who mine blocks are capable of altering the block information of the future block within a short time interval (roughly 900 seconds) [40].

⁴Resources external to the blockchain are considered as off-chain, while data stored on the blockchain is regarded as on-chain.

Solidity Source code	Instruction	Definition
Block Information		
<code>block.blockhash</code>	BLOCKHASH	Get the hash of one of the 256 most recent blocks
<code>block.coinbase</code>	COINBASE	Get the current block's beneficiary address
<code>block.timestamp</code>	TIMESTAMP	Get the current block's timestamp
<code>block.number</code>	NUMBER	Get the current block's number
<code>block.difficulty</code>	DIFFICULTY	Get the current block's difficulty
<code>block.gaslimit</code>	GASLIMIT	Get the current block's gas limit
<code>block.chainid</code>	CHAINID	Get the chain ID
Environmental Information		
<code>address(this)</code>	ADDRESS	Get address of currently executing account
<code>address(this).balance</code>	SELFBALANCE	Get balance of currently executing account
<code>msg.sender</code>	CALLER	Get caller address
<code>tx.gasprice</code>	GASPRICE	Get price of gas in transaction environment
<code>codesize()</code>	CODESIZE	Get size of code running in current account
<code>extcodesize()</code>	CALLDATASIZE	Get size of input data in transaction environment
<code>extcodehash()</code>	EXTCODEHASH	Get hash of an account's code

Fig. 2. Block information and environmental information in Ethereum, where more details can be found at [36].

Malicious miners may thus manipulate the block information to generate a pseudo-random number that is beneficial to them.

Environmental Information. Common environmental information in Ethereum includes the address of a caller who issues the transaction, the gas price, the residual amount of gas, the size of the code running in the current environment, and so on. These kinds of environmental information have a certain degree of uncertainty, which is why they are usually regarded as random seeds to generate pseudo-random numbers. However, environmental information can be easily accessed and manipulated by attackers using simple tricks [41]. For example, an attacker can derive the address of a caller contract by using the `CREATE2` instruction [42]. For the residual amount of gas, an attacker can measure how much the execution is going to use beforehand, and simulate the execution locally with an instrumented EVM, thereby knowing the residual amount of gas at a given moment in time during the execution.

Contract Member Variable. In smart contracts, the types of member variables are divided into public and private. External contracts cannot directly access private member variables in the contract. As such, developers consider taking private member variables as random seeds. However, information on the blockchain is completely public. As a result, anyone can get the values of private member variables in the contract by means of off-chain access. As an example, `web3.js` is a collection of libraries that allows users to interact with a local or remote Ethereum node [43]. With function `web3.eth.getStorageAt`, users have the capacity to gain access to any storage content of the contract.

Other On-Chain Information. Some developers may adopt the frequently-updating information in the contract as a random seed, like the total amount of token issuance in a token contract. Another consideration is to package the logic of pseudo-random number generation into a closed-source contract, reducing the risk of pseudo-random numbers being predicted. Nevertheless, for smart contracts, closed-source does not imply that the logic of the contract is hidden. Attackers are able to access the bytecode of the contract. Moreover, many decompilation tools, such as

Erays [44] and Gigahorse [45], can be used to help attackers understand the logic of the closed-source contract, so as to grasp the principles behind the generated pseudo-random numbers.

2) *Randomness Generated With Off-Chain Resources*: Our second insight is that smart contracts generate pseudo-random numbers with the assistance of off-chain resources. Considering that adversaries can easily manipulate on-chain information, developers resort to off-chain resources for safer pseudo-random number generation. After a detailed investigation, we found that using off-chain resources for pseudo-random number generation can be summarized into four types, such as oracle, verifiable random function, Commit-Reveal, and verifiable delay function. In what follows, we will elaborate on the details of these methods.

Oracle. Oracles are entities that connect the blockchain to external systems, allowing smart contracts to get access to off-chain resources like asset prices, token exchange rates, and random number seeds. For example, Provable [46], which serves as a popular oracle, provides random seeds to smart contracts by requesting external random number generation websites. Unfortunately, using oracles to generate pseudo-random numbers thus suffers from the inherent problem of whether the data obtained from oracles is trustworthy. An oracle with deprecated or malicious content could result in disastrous impacts on smart contract applications that employ the oracle information to generate pseudo-random numbers. For instance, a synthetic asset issuance platform *Syntheticx* experienced an oracle attack in 2019 [47], which leads to an incorrect conversion rate for sETH tokens, netting the attacker over 37 million sETH. In 2020, the decentralized smart contract application *bZx* was compromised by attackers [48], who exploited the price oracle manipulation to steal 2,378 ETH. These two cases are not isolated and many smart contracts fell victim to the oracle manipulation attack, including *yEarn* [49], *Harvest* [50], *Uniswap* [51], etc. As countermeasures, recent works such as *BLOCKEYE* [52] and *PROMUTATOR* [53] have explored detecting the oracle manipulation attack. While oracle attack detection techniques have been constantly investigated, it is still in its infancy and leaves a large room for progress.

Verifiable Random Function. Verifiable random function (VRF) is a public-key pseudo-random function that was initially put forth by Micali et al. [54]. Since then, VRF has witnessed widespread use in practice, such as lottery, cryptocurrency, and blockchain consensus. Given any input as the random seed, VRF will map the seed to a verifiable random number which is encoded with a private key signature. Public verifiers have the capacity to validate the authenticity and legitimacy of the pseudo-random number via the public key. As such, the pseudo-random number generated by VRF is unpredictable. Unfortunately, there exists the risk of collusion between the one who provides the seed input and the other who performs the signature, which threatens the security of generated verifiable random numbers. Chainlink [55], a well-known decentralized oracle, leverages verifiable random functions to generate cryptographically secure randomness on-chain, which can be used by smart contract developers as a source of the tamper-proof random-number generator.

Commit-Reveal. Commit-Reveal [56] is a multi-party random number generation scheme, which consists of two phases, i.e., *commit* and *reveal*. In the commit phase, every participant generates a random seed and calculates the corresponding hash value. All parties are required to send hash values to a proxy contract for safekeeping. In the reveal phase, each party hands over the generated random number seed to the proxy contract, which will verify whether the hash value of the seed matches the hash value submitted at the commit phase. Thereafter, random number seeds of all parties are combined to generate the final random number.

Commit-Reveal scheme ensures the unpredictability of generated pseudo-random numbers and prevents the leakage of random seeds to a certain extent. However, due to the openness and transparency of blockchains, the participant who is the last to submit the random seed can know the seeds disclosed by other parties, so that the final random number can be calculated in advance. As such, the last participant may refuse to submit his/her random number seed once the final random number is unfavorable to him/her, hindering the generation of random numbers.

Verifiable Delay Function. Verifiable delay function (VDF) is a function whose evaluation requires running a given number of sequential steps, yet the result can be efficiently verified [57]. VDF has been used to enhance some multi-party random number generation schemes. For example, for the Commit-Reveal, if the combined final random number is put into the VDF, and the time parameter of the VDF is set long enough (i.e., after the deadline of seed submission in the reveal phase), then even the last participant cannot foresee the result of the final random number. Moreover, VDF is useful for some scenarios which obtain random numbers from public sources. However, VDF is still in its infancy stage, enduring high overhead and latency.

B. Evaluating Random Number Generation Method

In this subsection, we present the evaluation results of different pseudo-random number generation strategies in Ethereum smart contracts. Specifically, we screen out 50 popular smart contracts such as *UniverseGalaxy* [29], *CryptoKitties* [31], and *Dice2Win* [32], and manually analyze the pseudo-random number generation methods adopted by these contracts. Quantitatively, 80% of smart contracts create pseudo-random numbers by using on-chain information, of which over 70% rely on block information, 8% utilize environmental information, and 18% focus on private member variables. On the other hand, the proportion of using off-chain resources to produce pseudo-random numbers accounts for 18%, of which 6% request off-chain oracles and 12% adopt the Commit-Reveal scheme to generate pseudo-random numbers. Note that a smart contract may employ multiple sources for producing pseudo-random numbers. As an example, contract *UniverseGalaxy* generates pseudo-random numbers by combining private member variables, caller address, and previous block hash.

Further, we propose eight indicators to evaluate the aforementioned pseudo-random number generation strategies. Table I shows the evaluation results of each method. From the table,

TABLE I
EVALUATION OF DIFFERENT RANDOM NUMBER GENERATION STRATEGIES. “—” DENOTES NOT APPLICABLE

Source	Method	Randomness	Unpredictable	Uncontrollable	Auditable	Open	Non-collusive	Cost	Latency
On-chain	Block	✓	✗	✓*	✓	✓	—	Low	Low
	Environmental	✓	✗	✗	✓	✓	—	Low	Low
	Member Variable	✓	✗	✓	✓	✓	—	Low	Low
Off-chain	Oracle	✓	✓	✓*	✗	✗	—	Mid	Mid
	VRF	✓	✓	✓	✓	✗	✗	Mid	Mid
	Commit-Reveal	✓	✓	✓	✓	✗	✓	Mid	Mid
	VDF	✓	✓	✓	✓	✓	✓	High	High

✓*: Generally speaking, random numbers generated using block information and oracles cannot be manipulated. Exceptions exist, miners who mine the block could manipulate the block information, while oracles are able to control the random number generated by themselves.

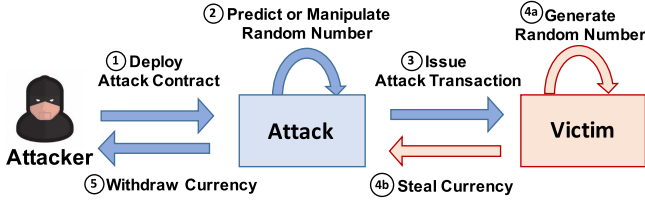


Fig. 3. The general process of a random number attack.

we observe that using on-chain information for generating a pseudo-random number has open and auditable nature, as well as low costs and latency. However, they still suffer from inherent problems of easily being predicted and even manipulated. In contrast, the pseudo-random number generated using off-chain resources is almost unpredictable and uncontrollable but leads to higher costs and latency. Particularly, using verifiable random functions for pseudo-random number generation may endure potential collusive risks. It is worth noting that all the investigated methods possess a certain degree of randomness since every strategy integrates hash algorithms.

C. Random Number Vulnerability and Attack

Definition 2 (Bad Randomness Vulnerability). When a smart contract relies on randomness (or a pseudo-random number) to generate a value or make a decision, but the sources of the randomness are not truly random or can be compromised or predicted by bad actors, a bad randomness vulnerability may occur.

Definition 3 (Random Number Attack). A random number attack refers to the exploitation of randomness vulnerabilities to predict or manipulate the generated pseudo-random number to steal illegal profits.

Upon reviewing the 50 popular smart contracts that have pseudo-random number generators, we know that most smart contracts tend to leverage on-chain information to produce pseudo-random numbers. As aforementioned, using merely on-chain information for pseudo-random number generation is insecure, which may leave a large room for security attacks. Generally, the overall process of the random number attack consists of five steps, which are depicted in Fig. 3.

- The attacker deploys an *Attack* contract on Ethereum;
- The *Attack* contract predicts or manipulates the generated pseudo-random number in a *Victim* contract;

```

1 contract Victim_1 {
2   uint256 private seed;
3   function lottery(uint256 number) payable public {
4     // generate random number
5     uint256 random = (block.timestamp + msg.sender + seed) % 10;
6     if (random == number && msg.value == 1 ether) {
7       // if a player guesses it, he/she obtains 9 ether
8       msg.sender.transfer(9 ether);
9     }
10  }
11 }
12 contract Attacker_1 {
13   // get the value of seed through off-chain access
14   function attack(Victim_1 victim, uint256 seed) public {
15     // predict random number
16     uint256 random = (block.timestamp + address(this) + seed) % 10;
17     // invoke guess function of contract Victim_1
18     victim.lottery{value: 1 ether}(random);
19   }
20 }

```

Fig. 4. A simplified example to show a transaction input manipulation attack.

- The *Attack* contract issues an invocation (i.e., attack transaction) to the *Victim* contract;
- The *Victim* contract generates a pseudo-random number and checks the invocation results of the *Attack* contract. If the invocation passes, the *Attack* contract steals digital currencies from the *Victim* contract;
- The attacker withdraws money from the *Attack* contract.

After investigating a variety of existing random number attacks, we summarize them into four categories, viz, transaction input manipulation, random number seed manipulation, direct prediction, and transaction rollback. It is worth noting that these four classes of random number attacks cover most of the attack situations. In the following, we will deliver concrete examples to illustrate each kind of random number attack.

1) Random Number Attack Classification: In this subsection, we take the *lottery* contract as an example to introduce each type of random number attack in detail.

Transaction Input Manipulation. Fig. 4 shows the simplified code of a gambling contract, which is written in Solidity. This contract realizes a lottery function that allows users to play by submitting their guesses along with participation fees. Function *lottery* combines the block timestamp *block.timestamp*, the caller address *msg.sender*, and the private member variable *seed* as the cardinal seed to generate the random number, i.e., variable *random*. The player who wants to participate in the game and submit a *guess* needs to pay 1 ether. If the *guess* number submitted by the player exactly matches the random number, the player will obtain 9 times the participation fee in return.

```

1 contract Victim_2 {
2   uint256 private seed;
3   function lottery() payable public {
4     uint256 random = (block.timestamp + msg.sender + seed) % 10;
5     if (random == 5 && msg.value == 1 ether) {
6       // if the player wins, he/she obtains 9 ether
7       msg.sender.transfer(9 ether);
8     }
9   }
10 }
11 contract Middleman {
12   function attack(Victim_2 victim) public {
13     victim.lottery{value: 1 ether}();
14   }
15 }
16 contract Attacker_2 {
17   function attack(uint256 seed) public {
18     // keep creating new middleman until the random is equal to 0
19     while(true) {
20       Middleman middle = new Middleman();
21       uint256 random = (block.timestamp + address(middle) + seed) % 10;
22       if (random == 5) {
23         middle.attack{value: 1 ether}();
24         break;
25       }
26     }
27   }
28 }

```

Fig. 5. A simplified example to show a random number seed manipulation attack.

Attack. The attacker deploys an attack contract *Attacker_1* to predict the random number. Due to the openness of Ethereum blockchain, the attacker can get the value of the private member variable *seed* in the victim contract *Victim_1* through off-chain access. Then, he/she uses the value as the *input* of the *attack* function in *Attacker_1* (line 14). After that, the function *attack* predicts the random number by combining the timestamp of the current block, its contract address (i.e., *address(this)*), and the variable *seed* (line 16), and then invokes the function *lottery* of contract *Victim_1* (line 18). Since smart contract transactions have the atomic nature (i.e., all operations in a transaction must be executed successfully, otherwise, the transaction reverts to its previous state), the entire process of function *attack* calling function *lottery* will be completed within the same transaction. Therefore, the block timestamps obtained by functions *attack* and *lottery* are identical. Furthermore, for function *lottery*, the address of the current contract caller (i.e., *msg.sender*) is exactly equal to the address of the contract *Attacker_1*. As a result, the attacker is able to successfully predict the random number generated by function *lottery*.

Random Number Seed Manipulation. We further present a updated gambling contract, as shown in Fig. 5. The contract implements a *lottery* function that users play by submitting only the participation fee of 1 ether. Function *lottery* combines the block timestamp, the caller address, and the private contract member variable to generate a random number, i.e., *random*. If *random* is exactly equal to 5, the player who participates in the current round lottery will obtain 9 times the participation fee in return (lines 5-7).

Attack. Since function *lottery* does not require any function inputs, it is impossible to predict the generated random number of function *lottery* by manipulating the transaction input. However, attackers could achieve the random number attack by directly manipulating the generation of the random number seed. First, the attacker takes the value of *seed* as the input (line 16). After that, the attacker creates an intermediary contract recurrently (line 19) until he/she finds an intermediate contract address so that the generated random number is equal

```

1 contract Victim_3 {
2   uint256 private seed;
3   function lottery() payable public {
4     uint256 random = (block.timestamp + msg.sender + seed) % 10;
5     if (random == 5 && msg.value == 1 ether) {
6       // if the player wins, he/she obtains 9 ether
7       msg.sender.transfer(9 ether);
8     }
9   }
10 }
11 contract Attacker_3 {
12   function attack(Victim_3 victim, uint256 seed) public {
13     uint256 random = (block.timestamp + address(this) + seed) % 10;
14     // attack Victim_3 only if profitable
15     if (random == 5) {
16       victim.lottery{value: 1 ether}();
17     } else {
18       return;
19     }
20 }

```

Fig. 6. A simplified example to show a direct prediction attack.

```

1 contract Victim_4 {
2   function lottery() payable public {
3     // random number using VRF is uncontrollable and unpredictable
4     uint256 random = verifiable_random_function();
5     if (random == 5 && msg.value == 1 ether) {
6       // if the player wins, he/she obtains 9 ether
7       msg.sender.transfer(9 ether);
8     }
9   }
10 }
11 contract Attacker_4 {
12   function attack(Victim_4 victim) public {
13     uint256 balanceBefore = this.balance;
14     victim.lottery{value: 1 ether}();
15     uint256 balanceAfter = this.balance;
16     // roll back the transaction if there is no profit
17     if (balanceAfter <= balanceBefore) {
18       revert;
19     }
20 }

```

Fig. 7. A simplified example to show a transaction rollback attack.

to 5 (line 21). In this attack, attackers can easily obtain the values of *block.timestamp* and *seed* (which is mentioned in the previous attack). Therefore, they are only required to control the caller address (i.e., *msg.sender*) to successfully predict the random number. It is worth mentioning that the well-known *Fomo3D* [13] contract was subjected to such kind of attack.

Direct Prediction. Besides manipulating the generation of random number seeds, an attacker can also pre-calculate random numbers before submitting a bet, and then place a bet only when he can make a profit. We present a direct prediction attack example in Fig. 6. Particularly, if the random number predicted by an attacker exactly matches the lottery number (line 14), the attacker will be rewarded with 9 times the participation fee (line 7). If the attacker fails to make a profit, he/she can stop the transaction early (line 17), thus avoiding losing their capital.

Transaction Rollback. Here, we further show an interesting random number attack. Sometimes the random number generation is complicated and unpredictable, e.g., using VRF to produce a random number. At this time, adversaries can exploit the transaction rollback mechanism to achieve a random number attack. As shown in Fig. 7, the attacker first checks the balance of his account (line 12). Then, the attacker invokes the function *lottery* of the victim contract *Victim_3* (line 13). After that, the attacker checks the account balance again (line 14), and determines whether he/she made a profit (line 16). If not, this transaction will be rolled back to the previous state, and the participation fee is given back to the attacker, thereby avoiding

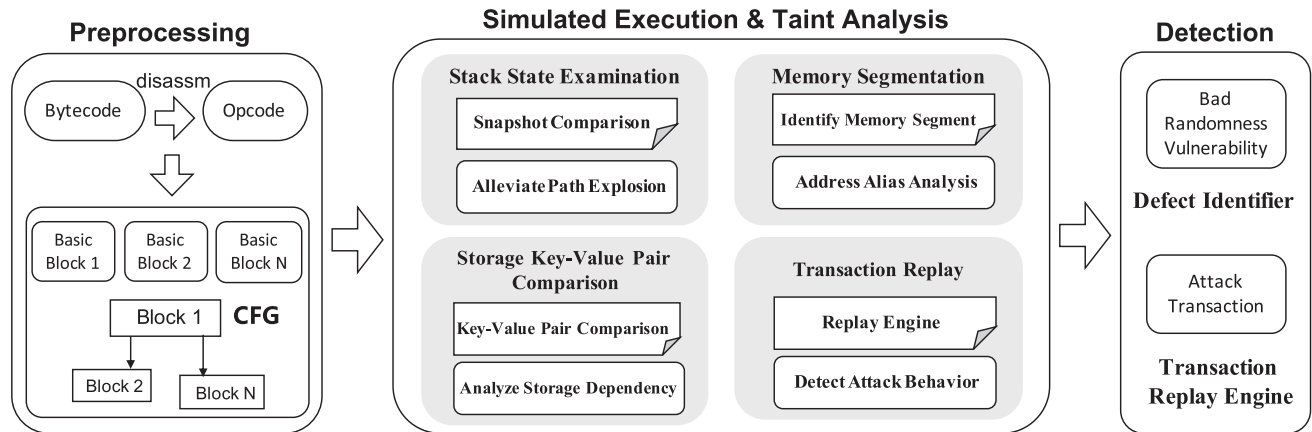


Fig. 8. A high-level overview of our pipeline. In the preprocessing phase, *RNVulDet* disassembles the bytecode of the smart contract into opcode and splits the opcode into a set of basic blocks for constructing the control flow graph. In the simulated execution phase, *RNVulDet* engages in four main components to facilitate taint analysis, giving the detection result of bad randomness vulnerabilities and random number attack transactions.

losses. Note that we have indeed detected many transaction rollback attacks from real-world Ethereum transactions in the experiments.

2) *Patterns for Detecting Random Number Attack*: In this subsection, we further define specific patterns to facilitate random number attack detection. Exactly, we design two types of attack patterns, where one is to identify the random number manipulation or prediction attack, while the other is to discover transaction rollback attack.

(1) Empirically, we observe that the first three types of random number attacks introduced in Section III-C1 can be concluded as *random number manipulation or prediction* attacks because they share a common pattern. That is, within a transaction, if a *caller* contract adopts the same vulnerable instructions (which will be defined in Section IV-A) of the *callee* contract to generate a pseudo-random number, and the random number generated in the *callee* contract contaminates the parameters of the Ether transfer instruction (i.e., *CALL*) or the conditional jump instruction (i.e., *JUMPI*) before the transfer instruction, the *caller* contract is considered to be an *attack* contract and this transaction issued by the *caller* contract is marked as an attack transaction (i.e., either random number manipulation or prediction).

(2) We detect the transaction rollback attack by checking if the rollback transaction and the profit-making transaction occur simultaneously within a short time. Particularly, we identify a rollback transaction by checking whether it has four steps: ① a *caller* contract issues an invocation to a *callee* contract; ② the *caller* contract queries its balance; ③ the balance information contaminates the subsequent conditional jump instruction (i.e., *JUMPI*); ④ the *caller* contract utilizes the *REVERT* instruction to rollback the current transaction.

Furthermore, we confirm a profit-making transaction by checking if it contains another four steps: ① a *caller* contract issues an invocation to a *callee* contract; ② the *callee* contract transfers money to the *caller* contract; ③ the *caller* contract queries its balance; ④ the balance information contaminates the subsequent conditional jump instruction (i.e., *JUMPI*).

IV. OUR METHOD

A. Overview of *RNVulDet*

Design Overview. Before diving into the details of our method, we outline the high-level pipeline of *RNVulDet*, which comprises preprocessing and simulated execution. Fig. 8 depicts the overall architecture of our proposed *RNVulDet*. In particular, *RNVulDet* engages in four main components to facilitate taint analysis, i.e., stack state examination, memory segmentation, storage key-value pair comparison, and transaction replay.

Preprocessing. Fundamentally, *RNVulDet* takes the bytecode of smart contracts as input. Solidity source code is also allowed, but it will be compiled to bytecode once it is fed into *RNVulDet*. Bytecode is then disassembled into opcode (short for operation code, also known as instruction code) by utilizing API provided by Geth [58]. *RNVulDet* splits the opcode into a set of basic blocks and generates the control flow graph (CFG) of the smart contract, which consists of basic blocks and control flow edges. The basic block usually comprises the sequence of instructions. *RNVulDet* will symbolically execute instructions in each block. One basic block is connected to subsequent basic blocks through a branch instruction in the CFG. In our analysis, we treat the branch instructions (e.g., *JUMP*, *JUMPI*) as the sign of the end of a basic block, which indicates that the branch instruction is considered a flag to segment the basic blocks. If the last instruction is a terminal instruction (e.g., *STOP*, *REVERT*, and *RETURN*), the block type is terminal [7]. Note that we actually do not recover the complete control flow graph of the contract bytecode, but rather restore the control flow execution paths from the bytecode, which is easier to implement than recovering a comprehensive control flow graph. For any given contract, *RNVulDet* is capable of extracting all control flow execution paths.

It is worth mentioning that detecting vulnerabilities on the bytecode level is very important for Ethereum smart contracts. All the bytecode of smart contracts are stored on the blockchain and can be readily accessed, but only a very small fraction (less than 1%) of smart contracts are open-sourced [59]. In particular,

smart contracts usually invoke external contracts, but the callee contract may not open its source code for inspection. In this context, the caller contract can only identify whether the callee contract is secure through the bytecode.

Simulated Execution. *RNVulDet* simulates the runtime environment of the Ethereum Virtual Machine (EVM), which symbolically executes each single EVM instruction one at a time. To satisfy the requirements of symbolic execution, *RNVulDet* simulates three kinds of storage structures, i.e., Stack, Memory, and Storage. During the execution, we can obtain the jump relations between basic blocks. *RNVulDet* executes the instructions in the basic blocks sequentially and generates the corresponding instruction instance (which refers to the execution result of instruction) for each instruction code. These instruction instances will be stored in the three types of storage structures.

Definition 4 (Vulnerable Instruction). Given the collection $\mathbb{T}$ that contains a set of EVM instructions, if any instruction in $\mathbb{T}$ used as a seed to generate pseudo-random numbers may introduce a bad randomness vulnerability, we say that every given instruction $t \in \mathbb{T}$ is vulnerable.

Specifically, *RNVulDet* executes basic blocks of the CFG in a depth-first traversal fashion. When the simulated execution encounters a *vulnerable instruction*,⁵ *RNVulDet* will mark it as the taint source, which is the headstream of taint propagation. Once the CALL instruction is triggered, the taint sink check will be performed to verify whether the parameters of the CALL instruction and the JUMPI instruction on the current execution path are contaminated by the taint source. Finally, *RNVulDet* analyzes the taint analysis results and reveals a bad randomness vulnerability with the assistance of a pattern analyzer. Note that *RNVulDet* is able to pinpoint the exact location of each detected bad randomness vulnerability. If a smart contract contains more than one bad randomness vulnerability, *RNVulDet* can reveal all of them as well as their exact locations. In the following, we show an example of how *RNVulDet* uses specific patterns to expose the bad randomness vulnerability.

Vulnerability Pattern. *RNVulDet* considers that a smart contract possesses a bad randomness vulnerability if the random number generated by *vulnerable instructions* contaminates the Ether transfer operation (i.e., CALL instruction). Specifically, we design three patterns to expose a bad randomness vulnerability. The first pattern CALLJUMPI checks if the *vulnerable instruction* affects the triggering condition of the transfer operation, namely whether the *vulnerable instruction* contaminates the JUMPI instruction which appears on the execution path of the CALL instruction. The second pattern is CALLToAddress which checks if the *vulnerable instruction* contaminates the transfer address, i.e., the parameter ToAddress of CALL instruction. The third pattern CALLValue concerns whether the *vulnerable instruction* contaminates the transfer amount, i.e., the parameter Value of CALL instruction.

⁵Notably, we focus on seven types of vulnerable instructions, namely, BLOCKHASH, COINBASE, DIFFICULTY, GASLIMIT, MOD_TIME, NUMBER, and TIMESTAMP.

RNVulDet reports that a function has a bad randomness vulnerability if it fulfills the combined pattern: CALLJUMPI $\wedge$ CALLToAddress $\wedge$ CALLValue.

B. Stack State Examination

Existing symbolic execution methods tend to traverse all execution paths as much as possible to expose possible vulnerabilities in smart contracts. However, an exhaustive traversal strategy introduces the inherent problem of path explosion, resulting in high performance overhead. Therefore, we consider proposing a method to alleviate the path explosion problem, which aims to reduce the execution of potential repeated program paths during taint analysis, improving the overall efficiency of simulated execution.

Rule 1: Snapshot Comparison. Given a snapshot S_1 and a historical snapshot S_h , we say that S_1 is equal to S_h iff all jump addresses and taint sources in S_1 are identical to those of S_h .

Technically, we design a stack state examination mechanism to prune potential invalid execution paths. *RNVulDet* will store the stack state snapshot for each basic block. Whenever the simulated execution reaches a basic block, *RNVulDet* performs the snapshot comparison (*Rule 1*) to determine whether the stack state snapshot of the block is equal to the one in the historical stack state snapshots. If it is matched, we consider that continuing along the current program path will not expose new vulnerabilities, thus the execution of this basic block and its subsequent paths will be skipped. Otherwise, the execution will continue along the current program trace. As such, *RNVulDet* can filter out possible repeated and bug-free paths, enforcing the simulated executor to trigger new traces and unearth potential bugs.

C. Memory Segmentation

In practice, *RNVulDet* simulates the executing environment of Memory to analyze memory data dependencies, namely read and write memory operands. However, since the operands of read and write instructions in Memory both are abstract instruction instances (a.k.a. the execution results of instructions), it is very difficult to judge whether the memory address accessed by read instruction is identical to that of write instruction. This will cause the read instruction to be unable to determine which instruction instance of a write instruction in Memory is its operand. It is worth noting that this is exactly the classical alias analysis puzzle in the field of program analysis. To address this problem, we propose a memory segmentation paradigm for handling read and write instruction dependencies in Memory.

To better understand the principle of memory segmentation, we first introduce how Solidity manages memory. Memory in the EVM is essentially an infinite array, which is addressed by bytes. In the traditional computer, the operating system is responsible for maintaining unused memory, allocating memory, and keeping track of memory that has been used. Unfortunately, there is no operating system in EVM, and thus memory allocation has to be completed by the Solidity compiler. Solidity currently adopts the free memory pointer (FMP) to mark the start of available addresses in Memory. Solidity always places new objects at the free memory pointer and memory is never freed,

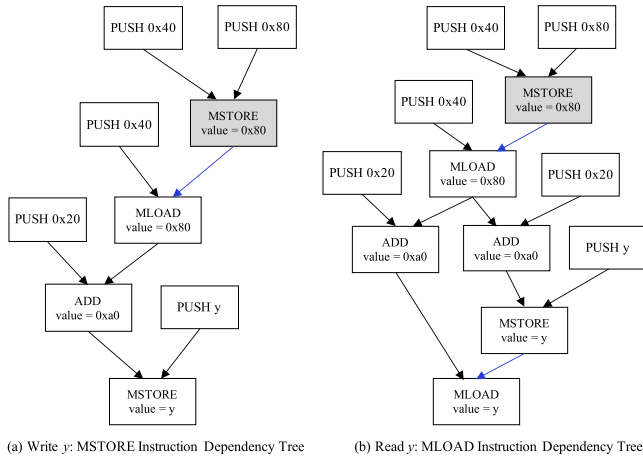


Fig. 9. Example: Data dependency analysis of MSTORE and MLOAD instructions in Memory. Node denotes the instruction instance. An arrow from node A to B indicates that A is the operand of B, where “ $\rightarrow$ ” represents the stack operand and “ $\dashrightarrow$ ” denotes the memory operand.

that is, the smart contract written by Solidity will not reclaim the memory that has already been allocated. As such, Solidity only allocates memory in segments incrementally. Once a new allocation demand occurs, it will allocate a section of memory space after the current FMP. Note that the initial FMP is stored in the memory space of 0×40 .

Inspired by this, we design a memory segmentation-based data dependency analysis paradigm. The core idea is to divide memory into several segments and map each read and write instruction to the corresponding memory segment one by one. When executing a read instruction, we take all instances of write instructions in the associated memory segment as the operands of the read instruction. Thereafter, the problem that we need to solve is how to mark each memory segment and how to identify the memory segment corresponding to read and write instructions.

Memory Segment Identification. According to the memory management strategy of Solidity, the FMP will be updated every time memory is allocated. We set the segment identifier at the location where each FMP is updated, respectively. In the meantime, whenever a memory allocation occurs, the write memory instruction MSTORE is executed. Therefore, we also set a unique identifier for each MSTORE. Moreover, to satisfy the memory allocation requirement, Solidity will allocate a section of memory after the current FMP. We can observe that the operand address of each read and write instruction instance is represented in the form of $\text{FMP} + \Delta$, where Δ represents the offset of the address of the written operand from that of the current FMP. Notably, the value of Δ is determined according to the specific location of the address to be written. As a result, we are able to utilize the FMP identifier and relevant offset to determine which memory segment the read and write instructions correspond to.

Memory Read and Write Dependency Analysis. Fig. 9 presents a specific example of memory read & write dependency analysis in *RNVulDet*. Technically, *RNVulDet* constructs the instruction dependency trees according to the address operand of read and write instruction instances in Memory, thereby determining which memory segment the instruction instance

is associated with. For MSTORE instruction dependency tree, *RNVulDet* first seeks out the MSTORE instruction with 0×40 as the address parameter, and then identifies the unique identifier of MSTORE instruction instance, so as to find the corresponding memory segment in Memory. Thereafter, *RNVulDet* places the instance of MSTORE instruction that writes value y in the memory segment. For MLOAD instruction dependency tree, *RNVulDet* finds the corresponding memory segment in Memory according to the unique identifier of MSTORE instruction instance, and then fetches all instruction instances in the memory segment as the memory operands of MLOAD instruction. Finally, *RNVulDet* extracts taint sources of the memory operand of MLOAD instruction instance that reads value y , thereby realizing the taint propagation in Memory.

D. Storage Key-Value Pair Comparison

Storage is organized as a key-value store, in which the instructions for reading and writing Storage are SLOAD and SSTORE, respectively. For analyzing data dependency in Storage, it is critical to determine whether the keys of SLOAD and SSTORE are identical. Yet, this is by no means easy. In addition, since data in Storage will be permanently stored in the Ethereum world state, it also involves the problem of cross-transaction taint propagation.

Key-Value Pair Comparison. We propose a novel key-value pair comparison strategy to address the data dependency analysis in Storage. During the simulation execution, whenever encountering the SSTORE instruction, *RNVulDet* will add the instruction instance to the end of Storage. When SLOAD is executed, *RNVulDet* searches forward from the end of Storage to find whether there is an instruction instance of SSTORE and whether the key of SSTORE is equal to that of SLOAD. Assuming there exists an instruction instance of SSTORE that has the same key as the instruction SLOAD, the instruction instance of SSTORE is used as the operand of SLOAD. Particularly, if multiple different SSTORE instructions execute on the same key, the value of the later SSTORE instruction will overwrite the value of the previous SSTORE instruction. Therefore, for the SLOAD instruction, it only needs to read the latest value of the key.

In practice, we found that the key of Storage represents a polynomial expression in the form of the sum of multiple instruction instances. Therefore, to facilitate the data dependency analysis of SSTORE and SLOAD in Storage, we consider flattening the instruction dependencies of the keys of SSTORE and SLOAD instructions into a *polynomial form*. In this context, determining whether the keys of SSTORE and SLOAD are equal in Storage is to judge whether each item of the SSTORE and SLOAD polynomials is a one-to-one correspondence. Note that the polynomial also contains function calls. The function with the same arguments may return different results since the values of some instructions can be changed during the simulation execution, e.g., PC, MSIZE, and GAS. Encouragingly, all the function calls in the polynomial will be included during the simulated execution. Therefore, in most cases, *RNVulDet* can ensure that when the polynomials of SSTORE and SLOAD are the same, they can return the same result.

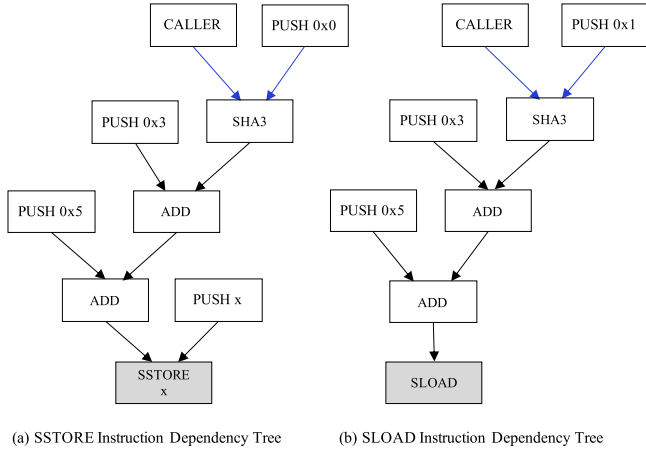


Fig. 10. Example: Data dependency analysis of SSTORE and SLOAD in Storage.

Fig. 10 presents a specific example to compare the key between the SSTORE instruction and the SLOAD instruction. *RNVulDet* constructs the instruction dependency trees of SSTORE and SLOAD, respectively. From the figure, we can observe that although the addition result (i.e., $0x3+0x5=0x8$) of the immediate value in the SSTORE instruction dependency tree is the same as that of the SLOAD, the operands of the hash instruction SHA3 are CALLER and PUSH 0x0 in SSTORE instruction tree while CALLER and PUSH 0x1 in SLOAD instruction tree. Therefore, *RNVulDet* can confirm that the key of the SLOAD instruction is different from that of the SSTORE instruction.

Cross-Transaction Taint Propagation. Unlike Stack and Memory which will reset the storage space after one contract execution, Storage will store data in the Ethereum world state for a long time. Therefore, when analyzing the data dependency of Storage, we do not only deal with taint analysis in the current contract execution but also further consider the problem of cross-transaction taint propagation in possible subsequent executions. For example, in the real-world game contract *FoMo3D* [13], when a user calls *FoMo3D* to play the game at the first invocation, the contract will record whether the user wins based on the matching result between the user's guess and the generated pseudo-random number of *FoMo3D*. If the user guesses it and calls *FoMo3D* to withdraw the reward at the second invocation, the contract determines whether to transfer the reward to the user according to matching results stored in Storage.

In this context, when the user calls the *FoMo3D* game contract first, the random number does not yet directly contaminate the *transfer* operation, but affects the matching result which will be stored in Storage. When the user invokes *FoMo3D* second, the matching result stored in Storage interferes with the *transfer* operation. Therefore, if the *FoMo3D* contract is only executed once by *RNVulDet*, the bad randomness vulnerability will not be exposed. To deal with such a problem, *RNVulDet* considers performing the contract many times. After every execution, the storage key-value pairs contaminated by *vulnerable instructions* will be retained, so that the subsequent execution could realize the cross-transaction taint propagation in Storage.

Algorithm 1: DYNAMIC TAINT ANALYSIS.

Input: $C_{target}, T_{suspicious}, C_{caller}, S_{vulnerable}$
Output: T_{attack}

```

1 executor.setup( $T_{suspicious.readSet}, T_{suspicious.input}$ );
2  $taint_{caller} \leftarrow set()$ ;
3  $taint_{target} \leftarrow set()$ ;
4 while executor.hasNextInst() do
5    $inst \leftarrow executor.getNextInst()$ ;
6   if  $inst \in S_{vulnerable}$  then
7      $taintSource(inst)$ ;
8    $taintPropagate(inst)$ ;
9   if  $inst == JUMPI \vee inst == CALL$  then
10     $taintSink(inst)$ ;
11 if  $taint_{caller} \cap taint_{target} \neq \emptyset$  then
12    $T_{attack} \leftarrow T_{suspicious}$ ;
13 Function taintSink ( $inst$ ):
14   if  $inst == JUMPI$  then
15     if  $executor.currentContract == C_{caller}$  then
16        $taint_{caller}.add(inst.taints)$ ;
17     else
18        $taint_{target}.add(inst.taints)$ ;
19   if  $inst == CALL$  then
20     if  $inst.from == C_{caller} \wedge inst.to == C_{target}$  then
21        $taint_{caller}.add(inst.taints)$ ;
22     if  $inst.from == C_{target} \wedge inst.to == C_{caller}$  then
23        $taint_{target}.add(inst.taints)$ 

```

E. Transaction Replay

In this stage, *RNVulDet* turns to random number attack monitoring, which engages in a transaction replay technique to detect potential attack transactions.

To monitor the transactions at the runtime, *RNVulDet* engages in a replay engine that is capable of replaying arbitrary transactions on Ethereum. Following our previous work [60], we define corresponding rules to locate suspicious random number attack transactions, and more importantly, provide fine-grained states that are needed by the replay engine. After retrieving the historical states, the replay engine re-executes these suspicious transactions. During the execution, *RNVulDet* adopts the dynamic taint analysis technique to introspect the execution of transactions and monitor the runtime states of a smart contract, thereby determining whether the transaction possesses a random number attack behavior.

Following the attack patterns defined in Section III-C2, we introduce a dynamic taint analysis method to detect corresponding random number attacks, which is summarized in Algorithm 1. *RNVulDet* takes the target contract C_{target} , the suspicious transaction $T_{suspicious}$, the caller contract C_{caller} , and the collection $S_{vulnerable}$ that contains a set of *vulnerable instructions* as inputs, and then outputs the marked attack transaction T_{attack} . *RNVulDet* will initialize the taint aggregation set of the caller contract and the target contract as $taint_{caller}$ and $taint_{target}$, respectively. First, *RNVulDet* reads a collection of suspicious transactions from the Ethereum world state database and restores the execution environment of the transactions (line 1). Then, the loop from lines 4 to 10 marks taint sources and performs taint propagation. *RNVulDet* executes the contract instructions sequentially and continuously obtains new instructions through *executor.getNextInst*() (line 5). If an instruction belongs to $S_{vulnerable}$, the instruction is vulnerable and will be marked

as a taint source, i.e., `taintSource(inst)` (lines 6-7). The taint source will propagate as the sequence of program instructions is executed. When encountering a `JUMPI` instruction or a `CALL` instruction, the taint sink check is triggered, i.e., `taintSink(inst)` (lines 9-10). When the execution of the suspicious transaction is completed, *RNVulDet* computes the intersection between $taint_{caller}$ and $taint_{target}$. If the intersection is not empty, *RNVulDet* will mark the suspicious transaction $T_{suspicious}$ as the real attack transaction T_{attack} .

More specifically, for instructions `JUMPI` and `CALL`, *RNVulDet* adopts different taint sink check strategies, respectively. (1) If the `JUMPI` instruction exists in the caller contract C_{caller} , the taint source $inst.taints$ that contaminates this instruction is added to $taint_{caller}$ (lines 15-16), which indicates that the control flow of the caller contract is affected by *vulnerable instructions*. Conversely, if the instruction exists in the target contract C_{target} , $inst.taints$ will be added to $taint_{target}$ (line 18), which means that the control flow of the target contract is influenced by $S_{vulnerable}$. (2) If the `CALL` instruction indicates that C_{caller} invokes C_{target} , the taint source $inst.taints$ is added to $taint_{caller}$ (lines 20-21), which implies that C_{caller} uses the pseudo-random number generated by *vulnerable instructions* to call $taint_{target}$. On the contrary, if the `CALL` instruction represents that $taint_{target}$ calls C_{caller} , $inst.taints$ will be added to $taint_{target}$ (lines 22-23), which suggests that the transfer of $taint_{target}$ to C_{caller} is influenced by *vulnerable instructions*.

To be more specific, the dynamic taint analysis technique is fully applicable to the detection of the *random number manipulation or prediction* attack (i.e., the first three types of random number attacks described in Section III-C1). For the *transaction rollback* attack, *RNVulDet* will confirm if a transaction contains the four steps based on the pattern defined in Section III-C2, where the fourth step, namely, whether the balance information contaminates the conditional jump instruction (i.e., `JUMPI`), is determined by using Algorithm 1.

V. EXPERIMENT

In this section, we carry out extensive experiments to evaluate the performance of *RNVulDet*, seeking to address the following research questions.

- *RQ1*: Can *RNVulDet* effectively identify bad randomness vulnerabilities in Ethereum smart contracts? How is its performance against state-of-the-art tools?
- *RQ2*: How is the execution efficiency of *RNVulDet* in analyzing smart contracts compared with other methods?
- *RQ3*: How does *RNVulDet* perform in detecting random number attack transactions?

In what follows, we first introduce the experimental settings, followed by answering these questions one by one.

A. Experimental Settings

Datasets. For this study, we crafted three kinds of datasets for validating the usefulness and effectiveness of *RNVulDet* from multiple perspectives.

(1) *Dataset_1*. We first collected Ethereum smart contracts that have bad randomness vulnerabilities from different sources, including Ethereum official platform, CVE database, GitHub repositories, and blog posts that analyze smart contracts. In total,

we collected 34 smart contracts and manually confirmed that they indeed possess bad randomness vulnerabilities. Note that a smart contract may contain more than one bad randomness vulnerability. We use this dataset to evaluate the false negatives of the detection tools we studied.

(2) *Dataset_2*. We rounded up popular Ethereum smart contracts from DappRadar [61] in February 2022. Specifically, we screened out the top 100 decentralized applications according to the user transaction volume at that time. As a result, we collected a total of 214 popular smart contracts. Note that these popular contracts have been rigorously audited before launch. We thus default the 214 smart contracts that are free of bad randomness vulnerabilities. Then, in the experiments, we manually check the contracts that are reported to have a bad randomness vulnerability by each tested tool, so as to assess the false positives of the corresponding detection tools.

(3) *Dataset_3*. We collected historical transactions of smart contracts in the first 12 million blocks in Ethereum, with block timestamps ranging from July 2015 to May 2021. Explicitly, this dataset contains rich information of each transaction, including transaction hash, transaction sender address, transaction value, and calling parameters. We also recorded all world states during each transaction execution, such as the account balance of each block. In total, we collected 38,665,879 contract bytecode, as well as the corresponding addresses and historical transactions. With the help of our previous tool [60], we filtered out 4,617 potential victim contracts, 43,051 potential malicious contracts, and 49,951 suspicious transactions for experiments.

Baseline Tools. After empirically scrutinizing existing vulnerability detection tools for smart contracts, we found that two industrial-grade static tools could detect weak randomness vulnerabilities, i.e., *Slither* and *Mythril*. We select *Slither* v0.8.2 and *Mythril* v0.22.35. *Slither* [16] is a static analysis framework that converts smart contracts into an intermediate representation and applies static program analysis techniques to discover vulnerabilities. *Mythril* [34] is a static tool that relies on concolic analysis, taint analysis, and control flow analysis to detect smart contract vulnerabilities. Moreover, we also compare with one dynamic tool, i.e., *ConFuzzius*, which can detect the block dependency vulnerability, a main category of bad randomness vulnerabilities. *ConFuzzius* [27] adopts a hybrid fuzzing strategy for smart contract vulnerability detection. While the baseline tools are able to take care of multiple types of vulnerabilities, they merely identify a subset of bad randomness vulnerabilities, lacking detailed and fine-grained analysis of random number vulnerabilities and attacks. Furthermore, while the baseline tools can detect bad randomness vulnerabilities, they still struggle to detect random number attacks.

Evaluation Metrics. There are five measurements obtained from our experiments to demonstrate the performance of tools, i.e., True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN), and CPU running time. TP represents the number of data items for which the detection tool correctly predicts samples with vulnerabilities. TN is the number of samples that the tool successfully detected without vulnerabilities. FP represents the number of samples that were falsely detected as having vulnerabilities. FN refers to the number of samples that were wrongly detected as projects without vulnerabilities.

TABLE II

TRUE POSITIVE AND FALSE NEGATIVE OF EACH TOOL ON DATASET_1, WHERE FALSE NEGATIVE REPRESENTS THAT A TOOL FAILS TO REVEAL A BAD RANDOMNESS VULNERABILITY

Method	True Positive (TP)	False Negative (FN)	Error
Slither [16]	18	16	0
Mythril [34]	16	18	0
ConFuzzius [27]	7	12	15
RNVulDet	30	4	0

CPU running time indicates the CPU time spent to analyze the contracts.

All experiments were performed on a computer running Ubuntu 21.04 Server and equipped with two Intel(R) Xeon(R) Silver 4214 CPUs at 2.20 GHz and 128 GB of memory. For a fair comparison, we set a timeout of 3,600 seconds for each tool. When it comes to timeout, the vulnerability detection will be stopped while the vulnerabilities already found before the timeout will not be lost.

B. Experimental Results and Analysis

1) *Effectiveness (RQ1)*: First, we benchmark *RNVulDet* against existing vulnerability detectors on *Dataset_1*, which consists of 34 smart contracts that have bad randomness vulnerabilities. We present the true positives and false negatives of each tool. Experimental results are summarized in Table II. From the table, we can observe that *RNVulDet* significantly outperforms other tools, and existing methods have not yet achieved a high true positive rate. For example, for static tools, *RNVulDet* generates 30 true positives, 12 and 14 more than *Slither* and *Mythril* respectively, while yielding only 4 false negatives. Furthermore, the dynamic tool *ConFuzzius* only generates 7 true positives, 23 less than *RNVulDet*, respectively. Intuitively, for fuzzing tools, if they do not cover some program regions, they will not reveal bugs in those regions. It is worth noting that *ConFuzzius* experienced an error on 15 contracts of *Dataset_1* due to a problem within its contract deployment mechanism.

Further, we analyze the reasons for the 4 false negatives caused by *RNVulDet*. Among them, 2 false negatives are because the contract does not use *vulnerable instructions* to generate pseudo-random numbers while instead using simple arithmetic instructions, such as ADD, SUB, MUL, and DIV. Since these instructions are normal arithmetic instructions, our tool does not treat them as taint sources, thus reporting false negatives. The other 2 false negatives are because the generated pseudo-random number does not affect the Ether transfer. Explicitly, one of the false negatives is due to the fact that the generated pseudo-random numbers are used to mint NFTs (*a.k.a* Non-fungible tokens) [62], and the other is that the generated pseudo-random number affects other types of tokens rather than Ether. In summary, our system generates the 4 false negatives due to the fact that they fail to match the bad randomness vulnerability detection pattern, i.e., “*vulnerability instructions contaminate the Ether transfer*” (see *Vulnerability Pattern* in Section IV-A).

To further evaluate the effectiveness of *RNVulDet*, we examine it on *Dataset_2*, which consists of 214 smart contracts that are considered free of bad randomness vulnerabilities. Table III

TABLE III

FALSE POSITIVE AND TRUE NEGATIVE OF EACH TOOL ON DATASET_2, WHERE FALSE POSITIVE REPRESENTS THAT A TOOL FALSELY REPORTS A BAD RANDOMNESS VULNERABILITY

Method	False Positive (FP)	True Negative (TN)	Error
Slither [16]	21	187	6
Mythril [34]	29	185	0
ConFuzzius [27]	9	53	152
RNVulDet	4	210	0

TABLE IV

CPU RUNNING TIME OF EACH METHOD IN ANALYZING SMART CONTRACTS

Method	Average	Lower Quartile	Median	Upper Quartile
Slither [16]	3.34	0.79	1.41	2.11
Mythril [34]	2009.02	81.57	2785.44	3595.48
RNVulDet	2.98	0.06	0.07	0.12

demonstrates the number of false positives and true negatives produced by each tool, as well as the cases where the tool triggers a runtime error. Specifically, *RNVulDet* only generates 4 false positives, 21 and 29 less than *Slither* and *Mythril*. Moreover, *Slither* ran into a runtime error on 6 smart contracts. Through the manual review, we found that these errors are attributed to the internal breakdown of *Slither* rather than the problem of the experimental settings. For the dynamic tool, *ConFuzzius* produces 9 false positives. Since fuzzers detect vulnerabilities only when triggering abnormal results during fuzzing, *ConFuzzius* thus reports relatively few false positives. *ConFuzzius* also ran into errors on 153 contracts due to the contract deployment problem.

We also investigate the reasons behind the 4 false positives of our tool. Particularly, 2 false positives are because the contract uses a *future block hash* to generate pseudo-random numbers. It is worth mentioning that using a future block hash for pseudo-random number generation is generally considered safe. Since no one can know the content of a future block beforehand, it is almost impossible for miners to predict the pseudo-random number generated by a 256-bit future block hash. Our system produces false positives because BLOCKHASH is a vulnerable instruction. The other 2 false positives are because the contract uses a hybrid random number generation using “*vulnerable instructions + Commit-Reveal*”, which is immune to tampering by attackers or miners. Since the vulnerable instructions are included in the hybrid scheme, our system generates false positives. It is worth noting that some contracts without random number generators are mostly identified as true negatives. In addition, a pseudo-random number generated by using a verifiable delay function or a hybrid method such as “*commit-reveal + verifiable random function*” will also be considered as a true negative.

2) *Efficiency (RQ2)*: In this subsection, we systematically examine the efficiency of *RNVulDet* and compare it against other methods. We calculate the CPU running time of each tool on 248 smart contracts (i.e., the sum of *Dataset_1* and *Dataset_2*) and present the average execution time, lower quartile, median, and upper quartile, respectively. Quantitative experimental results of each tool are summarized in Table IV. From the table, we

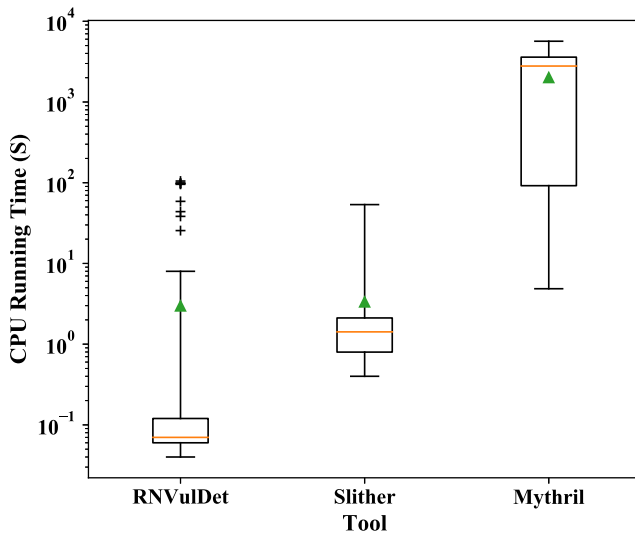


Fig. 11. Boxplot visualizes the CPU running time of each tool in analyzing smart contracts.

can observe that *RNVulDet* is significantly more efficient than others. For instance, the average execution time of *RNVulDet* is 2.98 s, 2,006.04 s faster than *Mythril*. It is worth noting that the dynamic tool *ConFuzzius* will keep running until the preset timeout is reached. As we set a timeout of 3,600 seconds for each tool, the average CPU running time of *ConFuzzius* is 3,600 seconds.

Further, we visualize the results of CPU running time for each tool in Fig. 11, which intuitively displays the distribution of CPU running time when analyzing different contracts. From the figure, we can learn that (1) The median (i.e., the orange line) of the boxplot for *RNVulDet* is near the bottom of the box, which indicates that *RNVulDet* takes a relatively short time to analyze smart contracts in most cases. (2) The rectangular area of the boxplot for *RNVulDet* is very small, which suggests that its execution time for different contracts is concentrated in a small range, thereby illustrating the stability of *RNVulDet*.

In particular, we also observe that the boxplot of *RNVulDet* contains 9 outliers (i.e., the cross symbol in Fig. 11), and the execution time is between 25.53 seconds and 104.92 seconds. By analyzing the detection results of *RNVulDet*, we found that all these outliers are generated when detecting the contracts of *FoMo3D* type. These outliers significantly drive up the average execution time of *RNVulDet* (i.e., triangles in Fig. 11). Note that although the detection time for the *FoMo3D* contract is relatively long, *RNVulDet* is still able to detect the bad randomness vulnerability. Instead, *Mythril* fails to reveal the bad randomness vulnerability in the *FoMo3D* contract due to the path explosion problem. Furthermore, *Slither* detects the bad randomness vulnerability by determining whether the random number seed is involved in a modulo operation, while the seed in the *FoMo3D* contract does not participate in a modulo operation, thus generating false negatives.

3) *Evaluation of Attack Detection (RQ3)*: We now present evaluation results on random number attack detection. We measure the effectiveness of *RNVulDet* by counting the number of

detected attack transactions. Particularly, we screen out the top 10 smart contracts with losses caused by random number attacks and visualize the relationship between losses and attack behaviors. Moreover, we calculate the period from when the contract was deployed to when a random number attack compromised it, providing overall insights on the exploitation of bad random vulnerabilities in Ethereum smart contracts.

First of all, we evaluate the performance of *RNVulDet* in detecting random number attack transactions on *Dataset_3*. Particularly, we find out potential attack transactions according to the defined attack patterns (which are defined in Section III-C2). Quantitative experimental results are summarized in Table V. From the table, we can observe that bad randomness vulnerabilities have caused a loss of 8,970 ETH (which is worth around \$27 million at the time of writing) from July 2015 to May 2021. To be more specific, a total of 265 attack contracts have launched 44,264 attack transactions, of which 42,107 belong to random number manipulation or prediction attacks. Note that we group the detected attack transactions by victim contracts and calculate the amount of Ether lost for each victim contract. We then rank the victim contracts in descending order of loss amount, and manually confirm the attack transactions of the contracts with relatively significant losses (more than 40 Ether), instead of all 44,264 attack transactions.

Further, we present the detection results of the top 10 contracts with losses caused by random number attacks in Table VI, where we can see that losses in the 10 contracts reached 8,467.89 ETH, accounting for 94.4% of the total losses of 181 detected victim contracts. Notably, *RNVulDet* successfully identifies bad randomness vulnerabilities in 9 of these contracts. Only contract *Dice2Win* was reported as a false positive. Through manual inspection, we found that *Dice2Win* adopts the hash of a future block to generate pseudo-random numbers. Although block hash (i.e., BLOCKHASH) is a kind of *vulnerable instruction*, using the *future block hash* as a random seed is very difficult for attackers to manipulate or predict. However, *RNVulDet* gives equal treatment to all *vulnerable instructions* and thus considers the pseudo-random number generated by a future block hash as vulnerable, resulting in a false alarm. Note that using other information of future blocks (e.g., *block timestamp*, *block number*, and *block difficulty*) to generate pseudo-random numbers still bears the inherent risk of being manipulated or predicted by miners. Only using a *future block hash* for random number generation is generally considered safe. Since no one can know the content of a future block beforehand, it is almost impossible for miners to predict the pseudo-random number generated by a 256-bit future block hash.

Fig. 12 further visualizes the relationship between losses and attack behaviors, where the blue and red nodes respectively represent victim contracts and attacker contracts, and the edges stand for attack behaviors. The blue nodes from 1 to 10 represent the top 10 victim contracts with the losses caused by random number attacks. The larger the area of the node, the greater the loss. The hollow red nodes denote the attackers who have compromised more than 10 contracts. From the figure, we obtain the following observations: 1) There are cases where multiple attackers attack the same victim contract, such as contracts *Last Winner* and *FoMo3D*, and 2) Some victim contracts, such

TABLE V
EXPERIMENT RESULT OF *RNVulDet* IN DETECTING RANDOM NUMBER ATTACK TRANSACTIONS

Attack Type	Loss	Victim Contract	Attack Contract	Attack Transaction
Transaction Input Manipulation				
Random Number Seed Manipulation	8,928 ETH	145	243	42,107
Direct Prediction				
Transaction Rollback	42 ETH	36	22	2,157
Total	8,970 ETH	181	265	44,264

TABLE VI
DETECTION RESULT OF THE TOP 10 SMART CONTRACTS WITH LOSSES CAUSED BY RANDOM NUMBER ATTACKS

#	Contract Address	Contract Name	Loss	Result	Open-Sourced
1	0xdd9fd6b6f8f7ea932997992bbe67eabb3e316f3c	Last Winner	6,543.59 ETH	TP	✗
2	0x47663541167ece0b96d9e5c60f9e470b2a20f598	LD3D	894.46 ETH	TP	✗
3	0xa62142888aba8370742be823c1782d17a0389da1	FoMo3D	615.27 ETH	TP	✓
4	0xd1ceeeefa68a6af0a5f6046132d986066c7f9426	Dice2Win	276.14 ETH	FP	✓
5	0x8a883a20940870dc055f2070ac8ec847ed2d9918	RatScam	96.63 ETH	TP	✓
6	0x29488e24cfdaa52a0b837217926c0c0853db7962	SuperCard	86.28 ETH	TP	✓
7	0xab577eae199d63af1aa8a03068d23c81fba0619	Unnamed	77.65 ETH	TP	✗
8	0x5fe5b7546d1628f7348b023a0393de1fc825a4fd	Roulette	70.00 ETH	TP	✓
9	0x9f35334c9dc3c66347d33558b7cfe800380391b5	Unnamed	43.91 ETH	TP	✗
10	0xd05dc25d8dad48fb9cf242d812d8fb4a653adb95	Unnamed	40.10 ETH	TP	✗

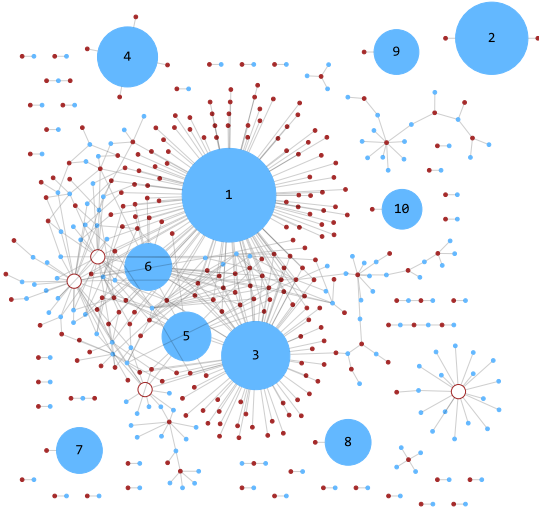


Fig. 12. Visual analysis of the relationship between losses and attack behaviors.

as *LD3D* and *Dice2Win*, have suffered from huge losses even though they were only attacked by a few attackers. Notably, taking *Fomo3D* as an example, we provide several real attack instances revealed by *RNVulDet*, namely, [63], [64], [65], and more detected attack transactions have been released on our Github.⁶

Finally, we depict the attack time window⁷ to illustrate the overall situation of the exploitation of bad random vulnerabilities in Ethereum smart contracts. Fig. 13 shows the attack time

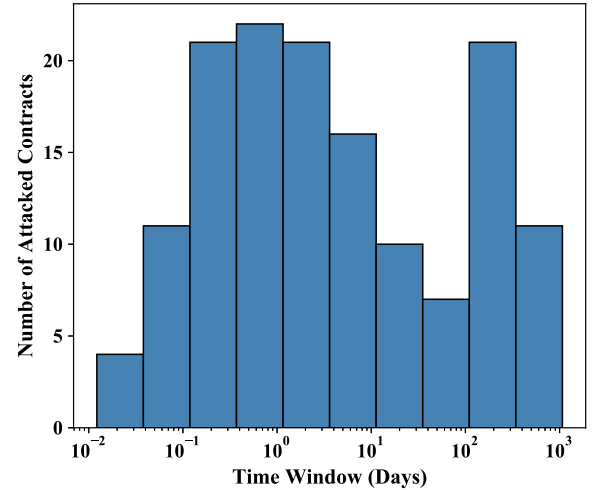


Fig. 13. Attack time window distribution of smart contracts with the bad randomness vulnerability.

window for 181 victim contracts that were compromised by random number attacks, where the shortest attack time window was 18 minutes. Put differently, it took only 18 minutes for a contract to be deployed on Ethereum until it was attacked. The longest attack time window was as high as 1,420 days. Moreover, there are 62 victim contracts whose attack time windows are less than 24 hours, which suggests the universality of random number attacks in Ethereum.

VI. DISCUSSION

In this section, we discuss the potential improvements of *RNVulDet* as well as its limitations and completeness.

Limitations and Completeness. (1) *RNVulDet* still faces a potential over-tainting problem during the taint analysis process.

⁶<https://github.com/Messi-Q/RNVulDet>

⁷Attack time window is defined as the period from when a contract was deployed to when a random number attack compromised it.

Since Solidity sometimes uses the exact same storage slot to store multiple variables, *RNVulDet* currently fails to differentiate the different variables across the same storage slot, which may lead to over-tainting and produce false alarms. (2) Regarding the polynomial comparison of *SSTORE* and *SLOAD* instructions, the function with the same arguments may return different results because the value of some instructions, such as *PC*, *MSIZE*, and *GAS*, may change during the simulation execution.

Future Improvement. (1) We will consider the impact of the shared storage slot and improve the polynomial comparison in the future to reduce the false positives and negatives. (2) We will optimize the bad randomness vulnerability patterns by taking care of the future block hash and hybrid random number generation, to eliminate possible false positives. Furthermore, it is also necessary to take into account the situations such as the random number generation using simple arithmetic instructions, and the generated random number contaminating the transfer of other types of tokens, thus reducing the false negatives. (3) We will extend the techniques presented in *RNVulDet*, such as Stack state examination, Memory segmentation, and Storage key-value pair comparison, to identify other types of smart contract vulnerabilities.

VII. RELATED WORK

Quite a few practitioners have devoted themselves to devising testing tools for smart contract vulnerability detection [10], [15], [16], [66], [67], [68]. Upon scrutinizing the released implementations of existing methods, we empirically found that current vulnerability detection approaches can be roughly divided into the following three major categories.

Static Analysis. Static analysis refers to a technology of analyzing programs without actually executing them. It examines code in the absence of real input data and is able to detect potential security violations, runtime errors, and logical inconsistencies. Oyente [15] is one of the pioneer smart contract analysis tools, which uses symbolic execution to identify smart contract vulnerabilities. Securify [22] performs advanced formal analysis to infer semantic facts of data flows in a smart contract and then prove the presence or absence of vulnerabilities. Slither [16] converts the smart contract source code into an intermediate representation of SlithIR. SlithIR uses a static single allocation form and a reduced instruction set to simplify the contract analysis process. Defectchecker [7] is a symbolic execution-based bug checker, which extracts the CFG of smart contracts and detects vulnerabilities by analyzing instruction sequences.

Fuzzing Testing. Fuzzing has been proven to be a prominent technique for discovering software vulnerabilities in the past decades [69]. When applied to smart contracts, a fuzzing engine will try to generate initial seeds to form executable transactions. The feedback of fuzzing results will dynamically guide the generation of new test cases. This process repeats until a stopping criterion is satisfied. Eventually, the fuzzer will analyze the results generated during fuzzing and report to users. Contract-fuzzer [40] is the first to apply fuzzing techniques to smart

contracts. It identifies vulnerabilities by monitoring runtime behaviors during fuzzing. ILF [25] proposes a new approach for learning an effective yet fast fuzzer from symbolic execution by phrasing the learning task in the framework of imitation learning. Harvey [70] extends standard greybox fuzzing with a method for predicting new inputs that are more likely to cover new paths or reveal vulnerabilities in smart contracts. sFuzz [17] presents an efficient and lightweight multi-objective adaptive strategy to dig out potential vulnerabilities hidden in those hard-to-cover branches.

Machine Learning. Recent years have witnessed an increasing practice of detecting program security vulnerabilities using machine learning techniques [71]. The advancement of machine learning has promoted the emergence of multiple vulnerability detection methods. Contractward [72] extracts bigram features from the smart contract opcode and adopts a variety of machine learning algorithms to detect bugs in smart contracts. S-gram [73] introduces a novel semantic-aware security auditing technique for analyzing smart contracts. The key insight behind S-gram is a combination of *N*-gram language modeling and lightweight static contract analysis. Liu et al. [19] propose to cast the source code of a smart contract into a contract graph and construct a temporal-message-propagation graph neural network to identify vulnerabilities. Xue et al. [18] present a machine learning-guided smart contract fuzzing framework, which leverages machine learning predictions to guide fuzzers for vulnerability detection.

VIII. CONCLUSION

In this work, we present a special focus on random numbers in Ethereum smart contracts. We investigate the principles behind different pseudo-random number generation strategies and organize them into a taxonomy. We propose *RNVulDet*, a tool that can automatically identify bad randomness vulnerabilities and detect random number attack transactions, with a very low false positive and negative rate. Specifically, *RNVulDet* consists of four key components, i.e., stack state examination, memory segmentation, storage key-value pair comparison, and transaction replay. An analysis of the real-world transactions in Ethereum revealed that 265 attackers have already issued 44,264 random number attack transactions, which caused more than \$27 million in losses. It is worth mentioning that these numbers solely provide a lower bound and thus might only reflect the tip of the iceberg. Experimental results on three kinds of datasets demonstrate that *RNVulDet* significantly surpasses state-of-the-art vulnerability detection tools by a large margin. Our implementation and dataset are released to facilitate future research. The presented designs in *RNVulDet* can also be transferable to detect other types of vulnerabilities in Ethereum smart contracts.

REFERENCES

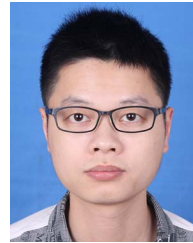
- [1] S. Ferretti and G. D'Angelo, "On the ethereum blockchain structure: A complex networks theory perspective," *Concurrency Computation: Pract. Experience*, vol. 32, no. 12, 2020, Art. no. e5493.

- [2] L. Zhao, S. Sen Gupta, A. Khan, and R. Luo, "Temporal analysis of the entire ethereum blockchain network," in *Proc. Web Conf.*, 2021, pp. 2258–2269.
- [3] T. Min, H. Wang, Y. Guo, and W. Cai, "Blockchain games: A survey," in *Proc. IEEE Conf. Games*, 2019, pp. 1–8.
- [4] Z. Jia, R. Chen, and J. Li, "DeLottery: A novel decentralized lottery system based on blockchain technology," in *Proc. 2nd Int. Conf. Blockchain Technol. Appl.*, 2019, pp. 20–25.
- [5] P. Qian, Z. Liu, X. Wang, J. Chen, B. Wang, and R. Zimmermann, "Digital resource rights confirmation and infringement tracking based on smart contracts," in *Proc. IEEE 6th Int. Conf. Cloud Comput. Intell. Syst.*, 2019, pp. 62–67.
- [6] M. Soni and D. K. Singh, "WITHDRAWN: Blockchain-based security & privacy for biomedical and healthcare information exchange systems," *Mater. Today: Proc.*, 2021. [Online]. Available: <https://doi.org/10.1016/j.matpr.2021.02.094>
- [7] J. Chen, X. Xia, D. Lo, J. Grundy, X. Luo, and T. Chen, "DefectChecker: Automated smart contract defect detection by analyzing EVM bytecode," *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2189–2207, Jul. 2022.
- [8] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural network," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 3283–3290.
- [9] P. Qian, Z. Liu, Q. He, R. Zimmermann, and X. Wang, "Towards automated reentrancy detection for smart contracts based on sequential models," *IEEE Access*, vol. 8, pp. 19 685–19 695, 2020.
- [10] D. Perez and B. Livshits, "Smart contract vulnerabilities: Vulnerable does not imply exploited," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1325–1341.
- [11] V. Dhillon, D. Metcalf, and M. Hooper, "The DAO hacked," in *Blockchain Enabled Applications*. Berlin, Germany: Springer, 2017, pp. 67–78.
- [12] S. Palladino, "The parity wallet hack explained," 2017. [Online]. Available: <https://blog.zeppelin.solutions/on-the-parity-wallet-multisig-hack-405a8c12e8f7>
- [13] SECBIT, "A comprehensive solution to bugs in Fomo3D-like games," 2021. [Online]. Available: <https://hackernoon.com/a-comprehensive-solution-to-bugs-in-fomo3d-like-games-ab3b054f3cc5>
- [14] Inspex, "Reentrancy attack on cream finance," 2021. [Online]. Available: <https://inspexco.medium.com/reentrancy-attack-on-cream-finance-incident-analysis-1c629686b6f5>
- [15] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 254–269.
- [16] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM 2nd Int. Workshop Emerg. Trends Softw. Eng. Blockchain*, 2019, pp. 8–15.
- [17] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, "sFuzz: An efficient adaptive fuzzer for solidity smart contracts," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng.*, 2020, pp. 778–788.
- [18] Y. Xue et al., "xFuzz: Machine learning guided cross-contract fuzzing," *IEEE Trans. Dependable Secure Comput.*, early access, Jun. 13, 2022, doi: [10.1109/TDSC.2022.3182373](https://doi.org/10.1109/TDSC.2022.3182373).
- [19] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, and X. Wang, "Combining graph neural networks with expert knowledge for smart contract vulnerability detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 1296–1310, Feb. 2023.
- [20] Z. Liu, P. Qian, X. Wang, L. Zhu, Q. He, and S. Ji, "Smart contract vulnerability detection: From pure neural network to interpretable graph feature and expert pattern fusion," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 2751–2759.
- [21] S. Kalra, S. Goel, M. Dhawan, and S. Sharma, "Zeus: Analyzing safety of smart contracts," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–12.
- [22] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 67–82.
- [23] S. So, M. Lee, J. Park, H. Lee, and H. Oh, "VERISMA: A highly precise safety verifier for Ethereum smart contracts," in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1678–1694.
- [24] J. S. Reis, P. Crocker, and S. M. de Sousa, "Tezla, an intermediate representation for static analysis of Michelson smart contracts," in *Proc. 2nd Workshop Formal Methods Blockchains*, 2020, pp. 4:1–4:12.
- [25] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 531–548.
- [26] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, and S. K. Cha, "SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses," in *Proc. IEEE/ACM 36th Int. Conf. Automated Softw. Eng.*, 2021, pp. 227–239.
- [27] C. F. Torres, A. K. Iannillo, A. Gervais, and R. State, "ConFuzzius: A data dependency-aware hybrid fuzzer for smart contracts," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2021, pp. 103–119.
- [28] Crytic, "Bad randomness," 2018. [Online]. Available: https://github.com/crytic/not-so-smart-contracts/blob/master/bad_randomness/README.md
- [29] UniverseGalaxy, "Universegalaxy," 2022. [Online]. Available: <https://dappradar.com/polygon/games/0xuniverse>
- [30] Fomo3D, "Fomo3D," 2022. [Online]. Available: <https://fomo3d.net/>
- [31] CryptoKitties, "Cryptokitties," 2022. [Online]. Available: <https://www.cryptokitties.co/>
- [32] Dice2Win, "Dice2win is a provably fair bets backed by simple open-sourced contract," 2022. [Online]. Available: <https://www.dapp.com/app/dice2win>
- [33] Z. Peng, "Not a fair game – fairness analysis of Dice2win," 2018. [Online]. Available: https://blogs.360.net/post/Fairness_Analysis_of_Dice2win_EN.html
- [34] B. Mueller, "A framework for bug hunting on the Ethereum blockchain," 2017. [Online]. Available: <https://github.com/ConsenSys/mythril>
- [35] N. Szabo, "Formalizing and securing relationships on public networks," *First Monday*, vol. 2, no. 9, Sep. 1997. [Online]. Available: <https://firstmonday.org/ojs/index.php/fm/article/view/548>
- [36] G. Wood et al., "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.
- [37] N. Grech, M. Kong, A. Jurisevic, L. Brent, B. Scholz, and Y. Smaragdakis, "MadMax: Surviving out-of-gas conditions in Ethereum smart contracts," *Proc. ACM Program. Lang.*, vol. 2, no. OOPSLA, pp. 1–27, 2018.
- [38] J. Clause, W. Li, and A. Orso, "Dytan: A generic dynamic taint analysis framework," in *Proc. Int. Symp. Softw. Testing Anal.*, 2007, pp. 196–206.
- [39] S. Ji, J. Dong, J. Qiu, B. Gu, Y. Wang, and T. Wang, "Increasing fuzz testing coverage for smart contracts with dynamic taint analysis," in *Proc. IEEE 21st Int. Conf. Softw. Qual. Rel. Secur.*, 2021, pp. 243–247.
- [40] B. Jiang, Y. Liu, and W. K. Chan, "ContractFuzzer: Fuzzing smart contracts for vulnerability detection," in *Proc. IEEE/ACM 33rd Int. Conf. Automated Softw. Eng.*, 2018, pp. 259–269.
- [41] D. He, Z. Deng, Y. Zhang, S. Chan, Y. Cheng, and N. Guizani, "Smart contract vulnerability analysis and security audit," *IEEE Netw.*, vol. 34, no. 5, pp. 276–282, Sep./Oct. 2020.
- [42] Alchemy, "How to use Create2 to deriving contract addresses," 2022. [Online]. Available: <https://docs.alchemy.com/lang-zh/docs/create2-an-alternative-to-deriving-contract-addresses>
- [43] Web3.js, "Web3.js - Ethereum JavaScript API," 2022. [Online]. Available: <https://web3js.readthedocs.io/en/v1.7.4/>
- [44] Y. Zhou, D. Kumar, S. Bakshi, J. Mason, A. Miller, and M. Bailey, "Erays: Reverse engineering Ethereum's opaque smart contracts," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1371–1385.
- [45] N. Grech, L. Brent, B. Scholz, and Y. Smaragdakis, "Gigahorse: Thorough, declarative decompilation of smart contracts," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng.*, 2019, pp. 1176–1186.
- [46] Provable, "The provable blockchain oracle for modern DApps," 2021. [Online]. Available: <https://provable.xyz/>
- [47] A. Shome, "Synthetix lost 37 million sETH in an Oracle attack," 2019. [Online]. Available: <https://www.financemagnates.com/cryptocurrency/news/synthetix-lost-37-million-seth-in-an-oracle-attack/>
- [48] PeckShield, "bZx hack II full disclosure (with detailed profit analysis)," 2020. [Online]. Available: <https://peckshield.medium.com/bzx-hack-ii-full-disclosure-with-detailed-profit-analysis-8126eccc1360>
- [49] S. Sun, "Accidentally stepping on a DeFi lego," 2020. [Online]. Available: <https://blog.trailofbits.com/2020/08/05/accidentally-stepping-on-a-defi-lego/>
- [50] REKT, "Harvest finance," 2020. [Online]. Available: <https://rekt.news/harvest-finance-rekt/>
- [51] ConsenSys, "Oracle manipulation - Ethereum smart contract best practices," 2020. [Online]. Available: <https://consensys.github.io/smart-contract-best-practices/attacks/oracle-manipulation/>
- [52] B. Wang et al., "BLOCKKEYE: Hunting for DeFi attacks on blockchain," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng.: Companion Proc.*, 2021, pp. 17–20.

- [53] S.-H. Wang, C.-C. Wu, Y.-C. Liang, L.-H. Hsieh, and H.-C. Hsiao, "ProMutator: Detecting vulnerable price Oracles in DeFi by mutated transactions," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops*, 2021, pp. 380–385.
- [54] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *Proc. IEEE 40th Annu. Symp. Found. Comput. Sci.*, 1999, pp. 120–130.
- [55] L. Breidenbach et al., "Chainlink 2.0: Next steps in the evolution of decentralized oracle networks," Chainlink Labs, 2021. [Online]. Available: <https://research.chain.link/whitepaper-v2.pdf>
- [56] H. Zhang, L.-H. Merino, V. Estrada-Galinanes, and B. Ford, "F3B: A low-latency commit-and-reveal architecture to mitigate blockchain front-running," 2022, *arXiv:2205.08529*.
- [57] B. Wesolowski, "Efficient verifiable delay functions," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 2019, pp. 379–407.
- [58] Go Ethereum, 2022. [Online]. Available: <https://github.com/ethereum/go-ethereum>
- [59] T. Chen et al., "TokenScope: Automatically detecting inconsistent behaviors of cryptocurrency tokens in Ethereum," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1503–1520.
- [60] S. Wu et al., "Time-travel investigation: Toward building a scalable attack detection framework on Ethereum," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, pp. 1–33, 2022.
- [61] DappRadar, "DappRadar," 2022. [Online]. Available: <https://dappradar.com/rankings/protocol/ethereum>
- [62] Wikipedia, "Non-fungible token," 2023. [Online]. Available: https://en.wikipedia.org/wiki/Non-fungible_token
- [63] Etherscan, "Fomo3D attack transaction," 2018. [Online]. Available: <https://etherscan.io/tx/0x10686f3a8222ac2870780fe7bfa15887c169ec33bdfa704c5938d3aa2bba44e6>
- [64] Etherscan, "Fomo3D attack transaction," 2018. [Online]. Available: <https://etherscan.io/tx/0x25fa30f545a8a43d257823400e81889348f265e58e42e932d072cb9f53315473>
- [65] Etherscan, "Fomo3D attack transaction," 2018. [Online]. Available: <https://etherscan.io/tx/0xee5942412508727cee6164d42f4b8122842eed765b59580cf9a145ac8e41bbf8>
- [66] L. Wu et al., "EthScope: A transaction-centric security analytics framework to detect malicious smart contracts on Ethereum," 2020, *arXiv:2005.08278*.
- [67] L. Lu et al., "iQuery: A trustworthy and scalable blockchain analytics platform," *IEEE Trans. Dependable Secure Comput.*, early access, Dec. 13, 2022, doi: [10.1109/TDSC.2022.3228908](https://doi.org/10.1109/TDSC.2022.3228908).
- [68] Z. Liu et al., "Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting," 2023, *arXiv:2301.03943*.
- [69] C. Chen, B. Cui, J. Ma, R. Wu, J. Guo, and W. Liu, "A systematic review of fuzzing techniques," *Comput. Secur.*, vol. 75, pp. 118–137, 2018.
- [70] V. Wüstholtz and M. Christakis, "Harvey: A greybox fuzzer for smart contracts," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 1398–1409.
- [71] C. Xing, Z. Chen, L. Chen, X. Guo, Z. Zheng, and J. Li, "A new scheme of vulnerability analysis in smart contract with machine learning," *Wirel. Netw.*, pp. 1–10, 2020. [Online]. Available: <https://doi.org/10.1007/s11276-020-02379-z>
- [72] W. Wang, J. Song, G. Xu, Y. Li, H. Wang, and C. Su, "ContractWard: Automated vulnerability detection models for Ethereum smart contracts," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1133–1144, Second Quarter 2021.
- [73] H. Liu, C. Liu, W. Zhao, Y. Jiang, and J. Sun, "S-gram: Towards semantic-aware security auditing for Ethereum smart contracts," in *Proc. IEEE/ACM 33rd Int. Conf. Automated Softw. Eng.*, 2018, pp. 814–819.



Peng Qian received the BE degree from Yangtze University, in 2018, and the ME degree from Zhejiang Gongshang University, in 2021. He is currently working toward the PhD degree with Zhejiang University. He has served as a program committee member for several conferences and journals such as IJCAI, ICME, and the *Multimedia Systems*. His current research interests include blockchain security and smart contract vulnerability detection. Various parts of his work have been published in first-tier venues including IJCAI, ICDE, WWW, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*.



Jianting He received the master's degree from Zhejiang University, China, in 2022. He is a researcher specializing in cyberspace security. His research currently focuses on smart contract security and DeFi security.



Lingling Lu is currently working toward the PhD degree with the College of Computer Science and Technology, Zhejiang University. She engages in blockchain security and optimization research work and has finished two blockchain application projects. Her research interests include blockchain, database managements, and Big Data processing.



Siwei Wu is currently working toward the PhD degree with Zhejiang University. His research currently focuses on smart contract security and DeFi security.



Zhipeng Lu is currently working toward the PhD degree with the College of Computer Science and Technology, Zhejiang University. His current research interests include smart contract security and DeFi security.



Lei Wu received the PhD degree in computer science from North Carolina State University, in 2015. He is currently an associate professor with Zhejiang University. His research interest lies mainly in security areas, including system security, and blockchain security.



Yajin Zhou received the PhD degree in computer science from North Carolina State University, in 2015. He is currently a ZJU100 young professor with Zhejiang University, China. His research spans software security, operating systems security, hardware-assisted security, and confidential computing. He is also interested in emerging areas, e.g., security of smart contracts, decentralized finance (DeFi) security, and underground economy.



Qinming He received the BE, MS, and PhD degrees in computer science from Zhejiang University, China, in 1985, 1988, and 2000, respectively. He is currently a professor with the College of Computer Science and Technology, Zhejiang University. His current research interests include blockchain security, machine learning, and data mining. He has published more than 90 papers in high-quality journals and conferences including ICDE, ACM CCS, IJCAI, SIGIR.